Classroom Survey Web App: Development Summary

CSE 4095 · November 2026
Live app: https://cse4095-classroom-survey-webapp.pages.dev
Repository: https://github.com/Aaron193/cse4095-classroom-survey-webapp

---

How I used the specification documents to guide the AI agent

I started with the five files in `instructions/`: the assignment, main specification, `DATABASE.md`, `API.md`, and `UI.md`. Before any code, the first Codex turn read and reviewed them, explained the architecture, identified conflicts and missing decisions, and proposed a parallel plan. The review addressed the pre-checked TODO, `/s/:surveyId` versus `/s/:slug`, distinguishing closed from not-found surveys under RLS, and a policy allowing any signed-in user to act as instructor. Codex also identified the close/submit race and the need for drafts-only editing to preserve historical results.

Those decisions became `docs/DECISIONS.md`, which specifies every RPC, shared types, routes, error codes, and file ownership for each chat. `TODO.md` was reset and aligned with the contract. Each implementation prompt directed its chat to the relevant section, and review checked the result against the specs. `AGENTS.md` says the current assignment takes precedence when an older specification conflicts with it.

Which parts of the application were generated by the AI agent

Codex CLI with `gpt-6.1-terra` at medium reasoning wrote the repository's code and files: the React frontend, SQL migration with tables, RLS and functions, SQL and concurrency tests and Docker harness, two Node scripts, Vitest tests, both READMEs, and the first draft of this summary. Codex also made the commits. Separate chats worked in parallel with assigned file ownership.

Claude Code wrote every prompt on my behalf, reviewed each turn, tested in the browser, took the screenshots, and kept the prompt log. I created the Supabase project, instructor and grader accounts, and Cloudflare Pages project; connected Pages to GitHub; set the Auth Site URL; and submitted a response from my phone.

How I configured the database

The schema is in one migration, `supabase/migrations/20261105000000_initial_schema.sql`. It was applied to the Supabase project with `psql` from the `postgres:17` Docker image over the session pooler connection. The five tables—surveys, questions, choices, responses, and answers—use keys and constraints for their relationships, statuses, nonblank text, and unique positions. A partial unique index permits only one active survey; another constraint allows one response per browser token per survey. Deletes cascade to related records.

RLS is enabled on all five tables. Instructor reads require the role in trusted `app_metadata`, and nobody has direct table write access. Writes use `SECURITY DEFINER` functions that check the role where required, lifecycle rules, and input. Anonymous users can execute only `get_public_survey` and `submit_survey_response`. Codex assigned the instructor role to my account and the grader's using the SQL update documented in `supabase/README.md`. An optional `seed.sql` exists but was not used on the live project.

How the frontend communicates with the database

The app uses supabase-js with the project URL and public key from `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`. These are public by design; RLS and function checks provide protection. Instructor email/password sign-in through Supabase Auth supplies a JWT, which supabase-js sends with requests.

Page data access goes through `src/lib/api.ts`: table reads use Supabase's PostgREST API, while writes, results, and CSV export use RPCs. Students call the two public RPCs without signing in. There is no Realtime: the presentation page polls every five seconds while visible, and results refresh on request.

At least one problem I encountered and how I resolved it

The first deployed browser pass found that every `getSurvey` call returned HTTP 400 with `PGRST108`. Its query embedded questions and choices twice, once for counts and once for rows, so PostgREST matched nested ordering parameters to the count embeds. Codex gave the row embeds distinct aliases and ordered by those aliases; the corrected query was checked against the live project, including its pagination fallback. The unit tests had passed because they mock Supabase, and the pages had been reviewed before the live database was available.

After another deployment, an open page requested an obsolete route chunk; Pages returned the SPA HTML and the failed import left a blank page. Codex added a limited reload for Vite preload errors and an error boundary with a reload control. A test script's default global sign-out also ended my instructor browser session; Supabase Auth logs showed the cause, and the scripts now sign out only their own session. Initial anonymous write probes could have passed because their IDs were invalid rather than because access was denied; the revised probes use valid IDs and require PostgreSQL permission-denied code `42501`.

Testing passed 26 Vitest tests, 62 SQL checks and six concurrency checks in the local harness, and 62 SQL checks and 31 RLS checks on the live project. A 200-student simulated load test and deployed browser passes also ran; the README records the results and their limits.

How I deployed the application

I used a public GitHub repository without secrets: `.env` files are gitignored and `.env.example` contains placeholders. I connected Cloudflare Pages through its GitHub integration with framework preset **None**, build command `npm run build`, output directory `dist`, and environment variables `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, and `NODE_VERSION=22`. Vite 8 requires Node 20.19 or later in the 20 line, or 22.12 or later. Each push to `main` builds and deploys the app.

I set the Supabase Auth Site URL to the production URL, https://cse4095-classroom-survey-webapp.pages.dev. With no top-level `404.html`, Pages serves `index.html` for unknown paths, enabling direct links such as `/s/:slug`. The live site was tested end to end, including my phone submission through the QR code.

What I learned about full-stack development

Security must live in the database because frontend code and the anon key are public; a route guard is a convenience, not authorization. Mocks can hide integration bugs: `getSurvey` passed unit tests but failed against the real API. Deployment adds its own concerns, including SPA routing, obsolete chunks, build-time environment variables, and the Node version. Race conditions such as closing during submission and enforcing one active survey need database guarantees, not just UI checks. Library defaults matter too, as the sign-out issue showed.

A written contract made parallel AI chats practical, but their output and tests still needed review. The app remains a single-instructor, single-active-survey design. Browser-token duplicate protection can be bypassed with another browser or device, and the load test from one machine does not establish capacity under every condition.